

QMSGR5074 - Project 3

Your Group ID: 1

Your UNIs: hi3921, lmw2141, rw3082

Your Full Names: Haoxuan Lu, Lindsay Varzarevsky, Rosanne Wei

Public GitHub Repo: [Projects-in-Advanced-Machine-Learning](#)

Part 1 – Data Ingestion & Preprocessing

1. Data Loading

- Acquire the Stanford Sentiment Treebank dataset.
- Split into training, validation, and test sets with stratified sampling to preserve class balance.
- Clearly document your splitting strategy and resulting dataset sizes.

```
# Import libraries and basic set up
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import tensorflow as tf
import random

# IMPORT DATA
!git clone https://github.com/YJiangcn/SST-2-sentiment-analysis.git

def seed_everything(seed=42):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)

seed_everything(42)

fatal: destination path 'SST-2-sentiment-analysis' already exists and is not an empty directory.
```

```
# Load train/dev
train_df = pd.read_csv(
    "SST-2-sentiment-analysis/data/train.tsv",
    sep="\t",
    header=None,
    names=["label", "sentence"]
)

dev_df = pd.read_csv(
    "SST-2-sentiment-analysis/data/dev.tsv",
    sep="\t",
    header=None,
    names=["label", "sentence"]
)

print("Train shape:", train_df.shape)
print("Dev shape:", dev_df.shape)

train_df.head()
```

Train shape: (6920, 2)		
Dev shape: (872, 2)		
label		sentence
0	1	a stirring , funny and finally transporting re...
1	0	apparently reassembled from the cutting-room f...
2	0	they presume their audience wo n't sit still f...
3	1	this is a visually stunning rumination on love...
4	1	jonathan parker 's bartleby should have been t...

```
# Check columns and labels
print("Train columns:", train_df.columns)
print("Dev columns:", dev_df.columns)

print("\nTrain label distribution:")
print(train_df["label"].value_counts())

print("\nDev label distribution:")
print(dev_df["label"].value_counts())

Train columns: Index(['label', 'sentence'], dtype='object')
Dev columns: Index(['label', 'sentence'], dtype='object')

Train label distribution:
label
1    3610
0     3310
Name: count, dtype: int64

Dev label distribution:
label
1     444
0     428
Name: count, dtype: int64
```

```
# Combine labeled data
full_df = pd.concat([train_df, dev_df], ignore_index=True)

full_df["label"] = full_df["label"].astype(int)

print("Full dataset shape:", full_df.shape)
print(full_df["label"].value_counts())

full_df.head()
```

Full dataset shape: (7792, 2)		
label		
1	4054	
0	3738	
Name: count, dtype: int64		
	label	sentence
0	1	a stirring , funny and finally transporting re...
1	0	apparently reassembled from the cutting-room f...
2	0	they presume their audience wo n't sit still f...
3	1	this is a visually stunning rumination on love...
4	1	jonathan parker 's bartleby should have been t...

```
# First split, hold out test
X_temp, X_test, y_temp, y_test = train_test_split(
    full_df["sentence"],
    full_df["label"],
    test_size=0.15,
    random_state=42,
    stratify=full_df["label"]
)

print("Temporary set size:", X_temp.shape)
print("Test set size:", X_test.shape)

Temporary set size: (6623,)
Test set size: (1169,)
```

```
# Split temporary set into train and validation
X_train, X_val, y_train, y_val = train_test_split(
    X_temp,
    y_temp,
    test_size=0.15,
    random_state=42,
    stratify=y_temp
)

print("Train set size:", X_train.shape)
print("Validation set size:", X_val.shape)
print("Test set size:", X_test.shape)

Train set size: (5629,)
Validation set size: (904,)
Test set size: (1169,)
```

```
# Convert splits into DataFrames
train_final = pd.DataFrame({
    "sentence": X_train,
    "label": y_train
})

val_final = pd.DataFrame({
    "sentence": X_val,
    "label": y_val
})

test_final = pd.DataFrame({
    "sentence": X_test,
    "label": y_test
})

train_final.head()
```


[illegible]

The preprocessor is saved as a pickle file so it can be reloaded later for inference. This ensures that new text inputs are cleaned, tokenized, and padded in the exact same way as the training data.

- **Traditional**: Build a TF-IDF vectorizer (or n-gram count) pipeline.
- **Neural**: Prepare sequences for embedding-pad/truncate to a fixed length.
- Save each preprocessor (vectorizer/tokenizer) to disk.

	sentence	label	clean_sentence
2932	human nature is a goofball movie , in the way ...	1	human nature is a goofball movie in the way th...
5920	build some robots , haul 'em to the theatre wi...	0	build some robots haul em to the theatre with ...
3206	with minimal imagination , you could restage t...	0	with minimal imagination you could restage the ...
6334	but no .	0	but no
2747	takes you by the face , strokes your cheeks an...	1	takes you by the face strokes your cheeks and ...

```
# Build TF-IDF vectorizer
tfidf_vectorizer = TfidfVectorizer(
    max_features=10000,
    min_df=2,
    ngram_range=(1, 2)
)

X_train_tfidf = tfidf_vectorizer.fit_transform(train_final["clean_sentence"])
X_val_tfidf = tfidf_vectorizer.transform(val_final["clean_sentence"])
X_test_tfidf = tfidf_vectorizer.transform(test_final["clean_sentence"])

print("Train TF-IDF shape:", X_train_tfidf.shape)
print("Validation TF-IDF shape:", X_val_tfidf.shape)
print("Test TF-IDF shape:", X_test_tfidf.shape)

Train TF-IDF shape: (5629, 10000)
Validation TF-IDF shape: (694, 10000)
Test TF-IDF shape: (1169, 10000)
```

```
Neural sequence feature shapes:
X_train_seq: (5629, 200)
X_val_seq: (994, 200)
X_test_seq: (1169, 200)

Labels:
y_train_seq: (5629,)
y_val_seq: (994,)
y_test_seq: (1169,)
```

```
# Reload TF-IDF vectorizer and test
with open("tfidf_vectorizer.pkl", "rb") as f:
    loaded_tfidf = pickle.load(f)

sample_tfidf = loaded_tfidf.transform([
    "This movie was surprisingly funny and emotionally powerful!"
])

print("Reloaded TF-IDF shape:", sample_tfidf.shape)

Reloaded TF-IDF shape: (1, 10000)
```

[illegible]

Both preprocessors were saved to disk using pickle. This allows the same TF-IDF vectorizer and neural tokenizer to be reloaded later for inference or deployment, ensuring that new text is processed in the same way as the training data.

1. Class Distribution

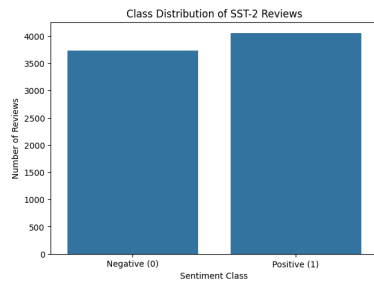
- Visualize the number of positive vs. negative reviews.
- Compute descriptive statistics on review lengths (mean, median, IQR).

```
# Import visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
# Visualize class distribution
label_counts = full_df["label"].value_counts().sort_index()

plt.figure(figsize=(7,5))
sns.barplot(
    x=["Negative (0)", "Positive (1)"],
    y=label_counts.values
)

plt.title("Class Distribution of SST-2 Reviews")
plt.ylabel("Number of Reviews")
plt.xlabel("Sentiment Class")
plt.show()
```



```
# Print class counts and proportions
print("Class Counts:")
print(full_df["label"].value_counts())

print("\nClass Proportions:")
print(full_df["label"].value_counts(normalize=True))
```

```
Class Counts:
label
1    4054
0    3738
Name: count, dtype: int64

Class Proportions:
label
1    0.528277
0    0.479723
Name: proportion, dtype: float64
```

```
# Create review length variables
full_df["word_count"] = full_df["sentence"].apply(lambda x: len(str(x).split()))
full_df["char_count"] = full_df["sentence"].apply(lambda x: len(str(x)))

full_df[["sentence", "word_count", "char_count"]].head()
```

	sentence	word_count	char_count
0	a stirring , funny and finally transporting re...	17	103
1	apparently reassembled from the cutting-room f...	12	78
2	they presume their audience wo n't sit still f...	35	225
3	this is a visually stunning rumination on love...	20	104
4	jonathan parker 's bartleby should have been l...	15	99

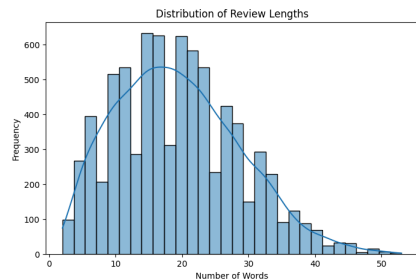
```
# Descriptive statistics for review lengths
mean_len = full_df["word_count"].mean()
median_len = full_df["word_count"].median()
q1 = full_df["word_count"].quantile(0.25)
q3 = full_df["word_count"].quantile(0.75)
iqr = q3 - q1
```

```
print("Review Length Statistics (Word Count)")
print("Mean : ", round(mean_len, 2))
print("Median : ", round(median_len, 2))
print("Q1 : ", round(q1, 2))
print("Q3 : ", round(q3, 2))
print("IQR : ", round(iqr, 2))
```

```
Review Length Statistics (Word Count)
Mean : 19.34
Median : 19.0
Q1 : 12.0
Q3 : 26.0
IQR : 14.0
```

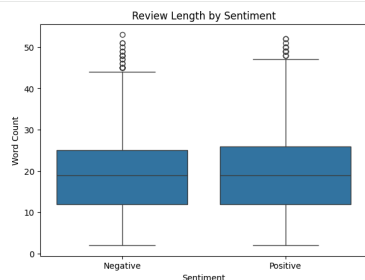
```
# Histogram of review lengths
plt.figure(figsize=(8,5))
sns.histplot(full_df["word_count"], bins=30, kde=True)

plt.title("Distribution of Review Lengths")
plt.xlabel("Number of Words")
plt.ylabel("Frequency")
plt.show()
```



```
# Boxplot of review lengths by sentiment
plt.figure(figsize=(7,5))
sns.boxplot(
    x=full_df["label"],
    y=full_df["word_count"]
)

plt.xticks([0,1], ["Negative", "Positive"])
plt.title("Review Length by Sentiment")
plt.xlabel("Sentiment")
plt.ylabel("Word Count")
plt.show()
```



The class distribution of the SST-2 dataset is relatively balanced, with similar numbers of negative and positive reviews. This is beneficial because balanced datasets reduce the risk of the model favoring one sentiment class over the other.

To understand text size, I calculated review length using word counts. The mean gives the average sentence length, while the median shows the typical review length after reducing the influence of outliers. I also computed the interquartile range (IQR), which measures the spread of the middle 50% of review lengths.

The histogram shows that most SST-2 reviews are short sentences, while the boxplot compares sentence lengths across positive and negative classes. This information helps guide sequence length selection for neural network models such as RNNs.

2. Text Characteristics

- Plot the 20 most frequent tokens per sentiment class.
- Generate word clouds (or bar charts) highlighting key terms for each class.

```
# Import token analysis libraries
from collections import Counter
from wordcloud import WordCloud
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
```

```
# Separate positive and negative reviews
negative_reviews = full_df[full_df["label"] == 0]["sentence"]
positive_reviews = full_df[full_df["label"] == 1]["sentence"]

print("Negative reviews:", len(negative_reviews))
print("Positive reviews:", len(positive_reviews))
```

Negative reviews: 3738
Positive reviews: 4054

```
# Create custom stopwords list (keep negation words)
stop_words = set(ENGLISH_STOP_WORDS) - {"not", "no", "never"}
```

```
# Clean and tokenize text with stopwords removal
neg_tokens = []
pos_tokens = []
```

```
for text in negative_reviews:
    words = preprocessor.clean_text(text).split()
    neg_tokens.extend([
        w for w in words
        if w not in stop_words and len(w) > 2
    ])

```

```
for text in positive_reviews:
    words = preprocessor.clean_text(text).split()
    pos_tokens.extend([
        w for w in words
        if w not in stop_words and len(w) > 2
    ])

```

```
print("Negative tokens:", len(neg_tokens))
print("Positive tokens:", len(pos_tokens))
```

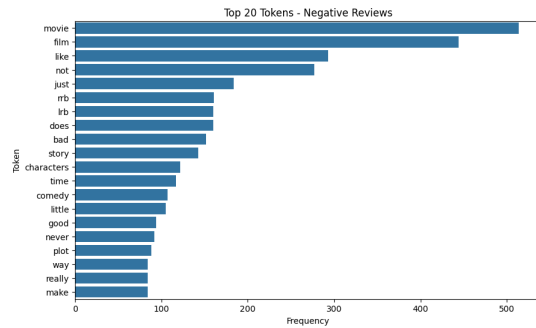
Negative tokens: 31645
Positive tokens: 36201

```
# Top 20 frequent negative tokens
neg_top20 = Counter(neg_tokens).most_common(20)

neg_words = [word for word, count in neg_top20]
neg_counts = [count for word, count in neg_top20]
```

```
plt.figure(figsize=(10,6))
sns.barplot(x=neg_counts, y=neg_words)
```

```
plt.title("Top 20 Tokens - Negative Reviews")
plt.xlabel("Frequency")
plt.ylabel("Token")
plt.show()
```

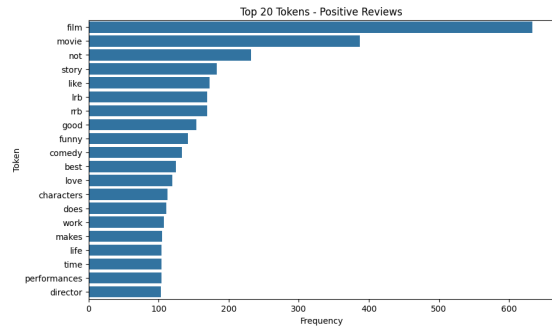


```
# Top 20 frequent positive tokens
pos_top20 = Counter(pos_tokens).most_common(20)

pos_words = [word for word, count in pos_top20]
pos_counts = [count for word, count in pos_top20]
```

```
plt.figure(figsize=(10,6))
sns.barplot(x=pos_counts, y=pos_words)
```

```
plt.title("Top 20 Tokens - Positive Reviews")
plt.xlabel("Frequency")
plt.ylabel("Token")
plt.show()
```



```
# negative word cloud
negative_text = " ".join(neg_tokens)
```

```
wordcloud_neg = WordCloud(
    width=900,
    height=500,
    background_color="white"
).generate(negative_text)
```

```
plt.figure(figsize=(12,6))
plt.imshow(wordcloud_neg, interpolation="bilinear")
plt.axis("off")
plt.title("Word Cloud - Negative Reviews")
plt.show()
```

Word Cloud - Negative Reviews



```
# Positive word cloud
positive_text = " ".join(pos_tokens)
```

```
wordcloud_pos = WordCloud(
    width=900,
    height=500,
    background_color="white"
).generate(positive_text)
```

[illegible]

	Negative Top Tokens	Positive Top Tokens
0	movie	film
1	film	movies
2	like	not
3	not	story
4	just	like
5	rb	lib
6	lr	rb
7	does	good
8	bad	funny
9	story	comedy

The word clouds provide a visual summary of the dominant vocabulary in each class. Larger words indicate higher frequency, making it easier to compare how language differs between positive and negative sentiment groups.

- Analyze whether review length correlates with sentiment.
- Present findings numerically and with at least one visualization.

		sentence	label	review_length
0	a stirring , funny and finally transporting re...	1	17	
1	apparently reassembled from the cutting-room f...	0	12	
2	they presume their audience wo n't sit still f...	0	35	
3	this is a visually stunning rumination on love...	1	20	
4	jonathan parker 's bartleby should have been t...	1	15	

	count	mean	median	std
Negative	3738	19.196094	19.0	9.164498
Positive	4054	19.478540	19.0	9.360632

```
# Boxplot of review length by sentiment
plt.figure(figsize=(8,5))
```

Review Length by Sentiment

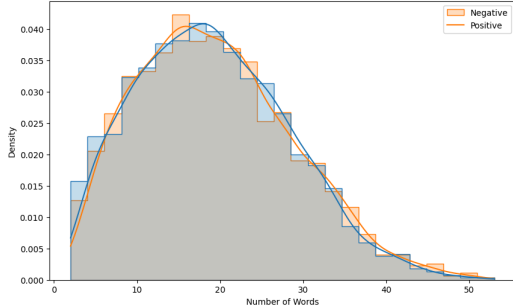
Sentiment	Min	Q1	Median	Q3	Max	Outliers
Negative	2	12	19	25	44	45, 46, 47, 48, 49
Positive	2	12	19	26	47	48, 49, 50, 51

```
# Histogram by sentiment
plt.figure(figsize=(10,6))

sns.histplot(
    data=full_df,
    x="review_length",
    hue="label",
    bins=25,
    kde=True,
    element="step",
    stat="density",
    common_norm=False
)

plt.title("Distribution of Review Length by Sentiment")
plt.xlabel("Number of Words")
plt.ylabel("Density")
plt.legend(labels=["Negative", "Positive"])
plt.show()
```

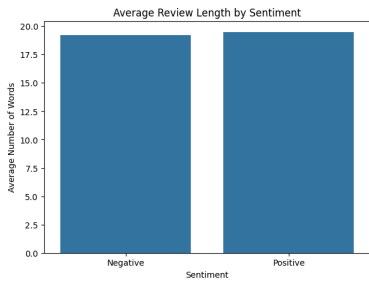
Distribution of Review Length by Sentiment



```
# Bar chart of average length
avg_lengths = full_df.groupby("label")["review_length"].mean()
plt.figure(figsize=(7,5))

sns.barplot(
    x=["Negative", "Positive"],
    y=avg_lengths.values
)

plt.title("Average Review Length by Sentiment")
plt.ylabel("Average Number of Words")
plt.xlabel("Sentiment")
plt.show()
```



To examine whether review length is related to sentiment, I calculated the number of words in each review and compared sentence lengths across negative and positive classes.

The numerical summary shows the average, median, and standard deviation of review length for each sentiment group. I also computed the Pearson correlation between sentiment label (0 = negative, 1 = positive) and review length. A correlation close to zero suggests that review length alone has only a weak relationship with sentiment.

The boxplot and histogram compare the distribution of sentence lengths between the two classes. While one class may have slightly longer reviews on average, there is substantial overlap between the groups. This suggests that review length may provide minor predictive information, but sentiment classification depends more heavily on the actual words and context than sentence length alone.

Part 3 – Baseline Traditional Models

1. Logistic Regression & SVM

- Train at least two linear models on your TF-IDF features (e.g., logistic regression, linear SVM).
- Use cross-validation (≥ 5 folds) on the training set to tune at least one hyperparameter.

```
# Import model libraries
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd
```

```
# Logistic Regression hyperparameter tuning (5-fold CV)
log_reg = LogisticRegression(
    max_iter=2000,
    random_state=42
)

log_params = {
    "C": [0.01, 0.1, 1, 5, 10]
}

log_grid = GridSearchCV(
    estimator=log_reg,
    param_grid=log_params,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

log_grid.fit(X_train_tfidf, y_train_seq)

print("Best Logistic Regression Parameter:")
print(log_grid.best_params_)

print("Best CV Score:")
print(round(log_grid.best_score_, 4))

Best Logistic Regression Parameter:
{'C': 10}
Best CV Score:
0.7934
```

```
# Best Logistic Regression model
best_log_reg = log_grid.best_estimator_

val_pred_log = best_log_reg.predict(X_val_tfidf)

print("Validation Accuracy:")
print(round(accuracy_score(y_val_seq, val_pred_log), 4))

Validation Accuracy:
0.7887
```

```
# Train Linear SVM with hyperparameter tuning
svm_model = LinearSVC(
    random_state=42,
    max_iter=5000
)

svm_params = {
    "C": [0.01, 0.1, 1, 5, 10]
}

svm_grid = GridSearchCV(
    estimator=svm_model,
    param_grid=svm_params,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

svm_grid.fit(X_train_tfidf, y_train_seq)

print("Best SVM Parameter:")
print(svm_grid.best_params_)

print("Best CV Score:")
print(round(svm_grid.best_score_, 4))

Best SVM Parameter:
{'C': 1}
Best CV Score:
0.7906
```

```
# Best Linear SVM model
best_svm = svm_grid.best_estimator_

val_pred_svm = best_svm.predict(X_val_tfidf)

print("Validation Accuracy:")
print(round(accuracy_score(y_val_seq, val_pred_svm), 4))

Validation Accuracy:
0.7867
```

```
# Compare validation performance
comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Linear SVM"],
    "Best C": [
        log_grid.best_params_["C"],
        svm_grid.best_params_["C"]
    ],
    "CV Accuracy": [
```

```
        ],
        svm_grid.best_score_,
        svm_grid.best_score_
    ],
    "Validation Accuracy": [
        accuracy_score(y_val_seq, val_pred_log),
        accuracy_score(y_val_seq, val_pred_svm)
    ]
})

comparison

# Classification report (Validation Set)
print("Logistic Regression Report")
print(classification_report(y_val_seq, val_pred_log))

print("\nLinear SVM Report")
print(classification_report(y_val_seq, val_pred_svm))
```

	Model	Best C	CV Accuracy	Validation Accuracy
0	Logistic Regression	10	0.793392	0.788732
1	Linear SVM	1	0.790550	0.786720

```
# Logistic Regression Report
precision    recall  f1-score   support

0           0.79    0.77    0.78     477
1           0.79    0.81    0.80     517

accuracy          0.79    0.79    0.79     994
macro avg         0.79    0.79    0.79     994
weighted avg      0.79    0.79    0.79     994

# Linear SVM Report
precision    recall  f1-score   support

0           0.79    0.76    0.77     477
1           0.78    0.81    0.80     517

accuracy          0.79    0.79    0.79     994
macro avg         0.79    0.79    0.79     994
weighted avg      0.79    0.79    0.79     994
```

```
# Visualize model comparison

plt.figure(figsize=(8,5))

sns.bargplot(
    data=comparison,
    x="Model",
    y="Validation Accuracy"
)

plt.title("Validation Accuracy Comparison")
plt.ylim(0.7, 1.0)
plt.show()
```



To establish baseline traditional models, I trained two linear classifiers using TF-IDF text features: Logistic Regression and Linear Support Vector Machine (SVM). Both models are widely used in sentiment classification because they perform well on sparse high-dimensional text data.

To improve performance, I used GridSearchCV with 5-fold cross-validation on the training set to tune the regularization parameter C. Cross-validation helps estimate how well the model generalizes to unseen data while reducing the risk of overfitting to a single split.

After selecting the best hyperparameters, I evaluated both models on the validation set. The comparison table reports cross-validation accuracy and validation accuracy. These models serve as strong baselines before comparing them with neural network architectures such as Simple RNN, LSTM, and GRU.

2. Random Forest & Gradient Boosting

- Train two tree-based models (e.g., Random Forest, XGBoost) on the same features.
- Report feature-importance for each and discuss any notable tokens.

```
# Import tree-based model libraries
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report
from xgboost import XGBClassifier
import numpy as np
import pandas as pd

# Train Random Forest model
rf_model = RandomForestClassifier(
    n_estimators=300,
    max_depth=25,
    min_samples_split=5,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train_tfidf, y_train_seq)

rf_val_pred = rf_model.predict(X_val_tfidf)

print("Random Forest Validation Accuracy:")
print(round(accuracy_score(y_val_seq, rf_val_pred), 4))

Random Forest Validation Accuracy:
0.7812
```

```
# Train Gradient Boosting model
# Convert sparse matrix to dense for Gradient Boosting
gb_model = XGBClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42,
    tree_method='hist',
    device='cuda'
)

gb_model.fit(X_train_tfidf, y_train_seq)

gb_val_pred = gb_model.predict(X_val_tfidf)

print("Gradient Boosting Validation Accuracy:")
print(round(accuracy_score(y_val_seq, gb_val_pred), 4))

Gradient Boosting Validation Accuracy:
0.5624
```

```
# Compare tree model performance
tree_results = pd.DataFrame({
    "Model": ["Random Forest", "Gradient Boosting"],
    "Validation Accuracy": [
        accuracy_score(y_val_seq, rf_val_pred),
        accuracy_score(y_val_seq, gb_val_pred)
    ]
})

tree_results

# Model Validation Accuracy
# Random Forest 0.7812
# Gradient Boosting 0.5624
```

```
# Random Forest classification report
print("Random Forest Report")
print(classification_report(y_val_seq, rf_val_pred))

print("\nGradient Boosting Report")
print(classification_report(y_val_seq, gb_val_pred))

# Random Forest Report
precision    recall  f1-score   support

0           0.76    0.55    0.64     477
1           0.67    0.84    0.75     517

accuracy          0.71    0.70    0.69     994
macro avg         0.71    0.70    0.69     994
weighted avg      0.71    0.70    0.69     994

# Gradient Boosting Report
precision    recall  f1-score   support

0           0.78    0.12    0.21     477
1           0.54    0.97    0.70     517

accuracy          0.66    0.55    0.46     994
macro avg         0.66    0.55    0.46     994
```



```
# Get TF-IDF feature names
feature_names = np.array(tfidf_vectorizer.get_feature_names_out())

print("Total Features:", len(feature_names))
```

Total Features: 10000

```
# Top 15 Random Forest important tokens
rf_importance = rf_model.feature_importances_

rf_idx = np.argsort(rf_importance)[-15:][::-1]

rf_top_words = feature_names[rf_idx]
rf_top_scores = rf_importance[rf_idx]

rf_importance_df = pd.DataFrame({
    "Token": rf_top_words,
    "Importance": rf_top_scores
})

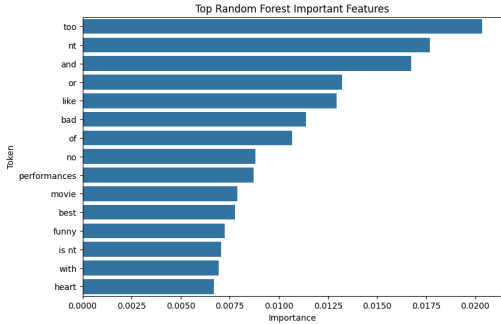
rf_importance_df
```

	Token	Importance
0	too	0.020341
1	nt	0.017693
2	and	0.016736
3	or	0.013212
4	like	0.012947
5	bad	0.011386
6	of	0.010657
7	no	0.008616
8	performances	0.008704
9	movie	0.007896
10	best	0.007773
11	funny	0.007233
12	is nt	0.007050
13	with	0.006929
14	heart	0.006701

```
# Visualize Random Forest importance
plt.figure(figsize=(9,6))

sns.barplot(
    data=rf_importance_df,
    x="Importance",
    y="Token"
)

plt.title("Top Random Forest Important Features")
plt.show()
```



```
# Top 15 Gradient Boosting important tokens
gb_importance = gb_model.feature_importances_

gb_idx = np.argsort(gb_importance)[-15:][::-1]

gb_top_words = feature_names[gb_idx]
gb_top_scores = gb_importance[gb_idx]

gb_importance_df = pd.DataFrame({
    "Token": gb_top_words,
    "Importance": gb_top_scores
})

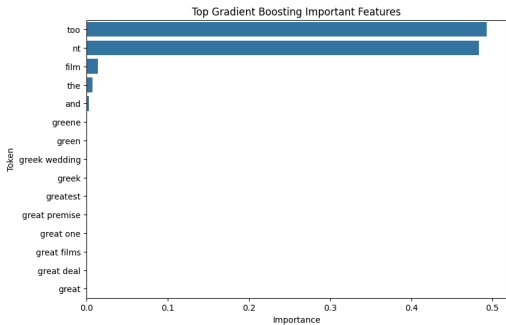
gb_importance_df
```

	Token	Importance
0	too	0.492672
1	nt	0.483080
2	film	0.014140
3	the	0.007413
4	and	0.002694
5	greene	0.000000
6	green	0.000000
7	greek wedding	0.000000
8	greek	0.000000
9	greatest	0.000000
10	great premise	0.000000
11	great one	0.000000
12	great films	0.000000
13	great deal	0.000000
14	great	0.000000

```
# Visualize Gradient Boosting importance
plt.figure(figsize=(9,6))

sns.barplot(
    data=gb_importance_df,
    x="Importance",
    y="Token"
)

plt.title("Top Gradient Boosting Important Features")
plt.show()
```



To compare non-linear traditional models, I trained two tree-based classifiers using the same TF-IDF features: Random Forest and Gradient Boosting. Tree-based models can capture non-linear interactions between tokens that linear models may miss.

The Random Forest model combines many decision trees built on random subsets of features, while Gradient Boosting builds trees sequentially to correct previous errors. Both models were evaluated on the validation set using accuracy and classification metrics.

Feature importance scores were extracted from each model to identify influential tokens. Highly ranked words often reflect strong positive or negative sentiment signals, such as evaluative adjectives or emotionally charged terms. These results improve interpretability by showing which words contribute most to classification decisions.

- Compute accuracy, precision, recall, F1-score, and ROC-AUC on the held-out test set.
- Present all results in a single comparison table.

```
# Import evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)
import pandas as pd
import numpy as np

# Generate test predictions for all models

# Logistic Regression
log_test_pred = best_log_reg.predict(X_test_tfidf)
log_test_prob = best_log_reg.predict_proba(X_test_tfidf)[:, 1]

# Linear SVM
svm_test_pred = best_svm.predict(X_test_tfidf)

# decision scores for ROC-AUC
svm_scores = best_svm.decision_function(X_test_tfidf)

# Random Forest
rf_test_pred = rf_model.predict(X_test_tfidf)
rf_test_prob = rf_model.predict_proba(X_test_tfidf)[:, 1]

# Gradient Boosting
X_test_dense = X_test_tfidf.toarray()
gb_test_pred = gb_model.predict(X_test_dense)
gb_test_prob = gb_model.predict_proba(X_test_dense)[:, 1]
```

```
# Create evaluation function
def evaluate_model(y_true, y_pred, y_score):
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred),
        "Recall": recall_score(y_true, y_pred),
        "F1-Score": f1_score(y_true, y_pred),
        "ROC-AUC": roc_auc_score(y_true, y_score)
    }
```

```
# Evaluate all models
results = {
    "Logistic Regression": evaluate_model(
        y_test_seq, log_test_pred, log_test_prob
    ),
    "Linear SVM": evaluate_model(
        y_test_seq, svm_test_pred, svm_scores
    ),
    "Random Forest": evaluate_model(
        y_test_seq, rf_test_pred, rf_test_prob
    ),
    "Gradient Boosting": evaluate_model(
        y_test_seq, gb_test_pred, gb_test_prob
    )
}
```

```
# Create comparison table
results_df = pd.DataFrame(results).T
results_df = results_df.round(4)

results_df
```

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.7613	0.7666	0.7780	0.7722	0.8456
Linear SVM	0.7588	0.7596	0.7845	0.7718	0.8427
Random Forest	0.6843	0.6583	0.8174	0.7293	0.7635
Gradient Boosting	0.5389	0.5304	0.9901	0.6908	0.5351

```
# Highlight best model per metric
best_metrics = results_df.idxmax()

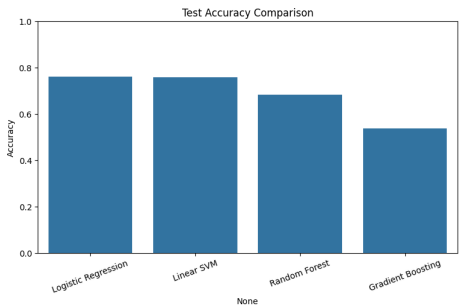
print("Best Model by Metric:")
print(best_metrics)
```

```
Best Model by Metric:
Accuracy      Logistic Regression
Precision      Logistic Regression
Recall        Gradient Boosting
F1-Score      Logistic Regression
ROC-AUC       Logistic Regression
dtype: object
```

```
# Visualization of model accuracy
plt.figure(figsize=(9,5))

sns.barplot(
    x=results_df.index,
    y=results_df["Accuracy"]
)

plt.title("Test Accuracy Comparison")
plt.xticks(rotation=20)
plt.ylim(0, 1.0)
plt.show()
```



To compare traditional machine learning models fairly, I evaluated all four models on the held-out test set using five common classification metrics: accuracy, precision, recall, F1-score, and ROC-AUC.

Accuracy measures the overall percentage of correct predictions. Precision reflects how often positive predictions were correct, while recall measures how many actual positive reviews were identified. F1-score balances precision and recall, making it useful when comparing classification quality. ROC-AUC evaluates ranking performance across decision thresholds.

The comparison table summarizes the performance of Logistic Regression, Linear SVM, Random Forest, and Gradient Boosting. This provides a clear benchmark before comparing these traditional approaches with neural network models such as Simple RNN, LSTM, and GRU.

Part 4 – Neural Network Models

1. Simple Feed-Forward

- Build an embedding layer + a dense MLP classifier.
- Ensure freeze vs. unfreeze embeddings in separate runs.

```
# Define uniformly within the global configuration section to ensure fairness in model experiment comparisons
from tensorflow.keras.callbacks import EarlyStopping

# Monitor validation set loss and restore the best weights.
global_early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,          # Stop if there is no progress for three consecutive rounds.
    restore_best_weights=True # Ensure that the final model obtained is the best one from the entire process.
)
```

```
# Simple Feed-Forward Neural Network

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Flatten, Dense
from tensorflow.keras.layers import Input
```

```
model_fnn = Sequential()

max_len = 200

model_fnn.add(Input(shape=(max_len,)))

model_fnn.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=32
    )
)
```

```
model_fnn.add(Flatten())
```

```
model_fnn.add(Dense(1000, activation="relu"))
model_fnn.add(Dense(32, activation="relu"))
model_fnn.add(Dense(1, activation="sigmoid"))
```

```
model_fnn.compile(
    optimizer="rmsprop",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
model_fnn.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 200, 32)	196,640
flatten_1 (Flatten)	(None, 6400)	0
dense_2 (Dense)	(None, 1000)	6,401,000
dense_3 (Dense)	(None, 32)	32,032
dense_4 (Dense)	(None, 1)	33

Total params: 6,629,705 (25.29 MB)
Trainable params: 6,629,705 (25.29 MB)
Non-trainable params: 0 (0.00 B)

```
history_fnn = model_fnn.fit(
    X_train_seq,
    y_train_seq,
    callbacks=[global_early_stop],
    epochs=20,
    batch_size=32,
    validation_data=(X_val_seq, y_val_seq),
    verbose=1
)
```

```
Epoch 1/20
176/176 — 19s 78ms/step - accuracy: 0.5566 - loss: 0.6812 - val_accuracy: 0.5895 - val_loss: 0.6583
Epoch 2/20
176/176 — 7s 41ms/step - accuracy: 0.7478 - loss: 0.5138 - val_accuracy: 0.6932 - val_loss: 0.6458
Epoch 3/20
176/176 — 8s 43ms/step - accuracy: 0.8957 - loss: 0.2548 - val_accuracy: 0.7344 - val_loss: 0.7035
Epoch 4/20
176/176 — 8s 44ms/step - accuracy: 0.9623 - loss: 0.1060 - val_accuracy: 0.7354 - val_loss: 0.8976
Epoch 5/20
176/176 — 8s 47ms/step - accuracy: 0.9854 - loss: 0.0418 - val_accuracy: 0.7193 - val_loss: 1.1150
```

```
fnn_val_loss, fnn_val_acc = model_fnn.evaluate(
    X_val_seq,
    y_val_seq,
    verbose=0
)
```

```
print("Feed-Forward NN Validation Accuracy:", round(fnn_val_acc, 4))
print("Feed-Forward NN Validation Loss:", round(fnn_val_loss, 4))
```

Feed-Forward NN Validation Accuracy: 0.6932
Feed-Forward NN Validation Loss: 0.6458

#same feed-forward model but with frozen embeddings:

```
model_fnn_frozen = Sequential()
```

```
model_fnn_frozen.add(Input(shape=(max_len,)))
```

```
model_fnn_frozen.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=32,
        trainable=False
    )
)
```

```
model_fnn_frozen.add(Flatten())
```

```
model_fnn_frozen.add(Dense(1000, activation="relu"))
model_fnn_frozen.add(Dense(32, activation="relu"))
model_fnn_frozen.add(Dense(1, activation="sigmoid"))
```

```
model_fnn_frozen.compile(
    optimizer="rmsprop",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
model_fnn_frozen.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 200, 32)	196,640
flatten_1 (Flatten)	(None, 6400)	0
dense_5 (Dense)	(None, 1000)	6,401,000
dense_6 (Dense)	(None, 32)	32,032
dense_7 (Dense)	(None, 1)	33

Total params: 6,629,705 (25.29 MB)
Trainable params: 5,425,065 (24.54 MB)
Non-trainable params: 196,640 (768.12 KB)

```
history_fnn_frozen = model_fnn_frozen.fit(
    X_train_seq,
    y_train_seq,
    callbacks=[global_early_stop],
    epochs=20,
    batch_size=32,
    validation_data=(X_val_seq, y_val_seq),
    verbose=1
)
```

```
Epoch 1/20
176/176 — 3s 9ms/step - accuracy: 0.5115 - loss: 0.6939 - val_accuracy: 0.5201 - val_loss: 0.6927
Epoch 2/20
176/176 — 1s 4ms/step - accuracy: 0.5203 - loss: 0.6925 - val_accuracy: 0.5201 - val_loss: 0.6932
Epoch 3/20
176/176 — 1s 4ms/step - accuracy: 0.5184 - loss: 0.6926 - val_accuracy: 0.5201 - val_loss: 0.6929
```

```
fnn_frozen_val_loss, fnn_frozen_val_acc = model_fnn_frozen.evaluate(
    X_val_seq,
    y_val_seq,
    verbose=0
)
```

```
print("Frozen Feed-Forward NN Validation Accuracy:", round(fnn_frozen_val_acc, 4))
print("Frozen Feed-Forward NN Validation Loss:", round(fnn_frozen_val_loss, 4))
```

Frozen Feed-Forward NN Validation Accuracy: 0.5201
Frozen Feed-Forward NN Validation Loss: 0.6927

```
fnn_results = pd.DataFrame({
    "Model": ["Feed-Forward NN", "Frozen Feed-Forward NN"],
    "Validation Accuracy": [fnn_val_acc, fnn_frozen_val_acc],
    "Validation Loss": [fnn_val_loss, fnn_frozen_val_loss]
})
```

fnn_results

	Model	Validation Accuracy	Validation Loss
0	Feed-Forward NN	0.693169	0.645814
1	Frozen Feed-Forward NN	0.520121	0.692064

```
import tensorflow.keras.backend as K
import gc
```

```
if 'global_predictions' not in locals():
    global_predictions = {}
```

```
def finalize_keras_model(name, model_obj, X_test, store_dict):
```

```
    """
    For Keras models: make predictions, store results, and clear memory.
    """
    print(f"Processing {name} model...")
```

```
    # 1. Perform a prediction and convert it into a class index (0 or 1).
    # Since the output layer uses a Sigmoid function, 0.5 is used as the threshold.
    y_probs = model_obj.predict(X_test, verbose=0)
    y_preds = (y_probs > 0.5).astype("int32").flatten()
```

```
    # 2. Store Prediction Results
    store_dict[name] = y_preds
    print(f"🟢 {name} prediction results have been stored in the dictionary")
```

```
    # 3. Completely remove the model from system memory and VRAM.
```

```
    try:
        del model_obj
        K.clear_session()
        gc.collect()
        import torch
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
    except:
        pass
```

```
    print(f"🔚 {name} Memory cleanup complete. \n")
```

```
# Clean FNN
if 'model_fnn' in locals():
    finalize_keras_model('Feed-Forward NN', model_fnn, X_val_seq, global_predictions)
```

```
Processing Feed-Forward NN model...
🟢 Feed-Forward NN prediction results have been stored in the dictionary
🔚 Feed-Forward NN Memory cleanup complete.
```

```

Clean FNN-Frozen
if model_fnn_frozen' in locals():
    finalize_keras_model('Frozen Feed-Forward NN', model_fnn_frozen, X_val_seq, global_predictions)

Processing Frozen Feed-Forward NN model...
✔ Frozen Feed-Forward NN prediction results have been stored in the dictionary
✔ Frozen Feed-Forward NN Memory cleanup complete.

```

The feed-forward neural network with trainable embeddings achieved substantially higher validation accuracy 0.69 compared to the frozen embedding model 0.52.

This suggests that allowing the embedding layer to update during training enabled the model to learn task-specific semantic representations that improved sentiment classification performance.

In contrast, the frozen embedding model was unable to adapt word representations to the dataset, leading to performance near random guessing.

Although the trainable model produced a higher validation loss, this likely reflects a small number of highly confident incorrect predictions, since binary cross-entropy strongly penalizes confident errors.

2. Convolutional Text Classifier

- Implement a 1D-CNN architecture (Conv + Pooling) for sequence data.
- Justify choice of kernel sizes and number of filters.

```
# Convolutional Text Classifier

from tensorflow.keras.layers import Conv1D, GlobalMaxPooling1D

```

```
model_cnn = Sequential()

model_cnn.add(Input(shape=(max_len,)))

model_cnn.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=32
    )
)

model_cnn.add(
    Conv1D(
        filters=64,
        kernel_size=5,
        activation="relu"
    )
)

model_cnn.add(GlobalMaxPooling1D())

model_cnn.add(Dense(64, activation="relu"))
model_cnn.add(Dense(1, activation="sigmoid"))

model_cnn.compile(
    optimizer="rmsprop",
    loss="binary_crossentropy",
    metrics=['accuracy']
)

model_cnn.summary()

```

Model: "sequential"		
Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 32)	196,640
conv1d (Conv1D)	(None, 196, 64)	10,304
global_max_pooling1d (GlobalMaxPooling1D)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dense_1 (Dense)	(None, 1)	65

Total params: 211,169 (824.88 KB)
Trainable params: 211,169 (824.88 KB)
Non-trainable params: 0 (0.00 B)

```
import time

start_time = time.time()

history_cnn = model_cnn.fit(
    X_train_seq,
    y_train_seq,
    callbacks=[global_early_stop],
    epochs=20,
    batch_size=32,
    validation_data=(X_val_seq, y_val_seq),
    verbose=1
)

cnn_train_time = time.time() - start_time

Epoch 1/20
176/176 ————— 25s 84ms/step - accuracy: 0.5649 - loss: 0.6814 - val_accuracy: 0.6368 - val_loss: 0.6399
Epoch 2/20
176/176 ————— 8s 46ms/step - accuracy: 0.7572 - loss: 0.5230 - val_accuracy: 0.7506 - val_loss: 0.5069
Epoch 3/20
176/176 ————— 9s 48ms/step - accuracy: 0.8511 - loss: 0.3490 - val_accuracy: 0.7777 - val_loss: 0.4741
Epoch 4/20
176/176 ————— 8s 46ms/step - accuracy: 0.8984 - loss: 0.2463 - val_accuracy: 0.7867 - val_loss: 0.4040
Epoch 5/20
176/176 ————— 8s 43ms/step - accuracy: 0.9343 - loss: 0.1723 - val_accuracy: 0.7907 - val_loss: 0.5226
Epoch 6/20
176/176 ————— 7s 37ms/step - accuracy: 0.9572 - loss: 0.1133 - val_accuracy: 0.7847 - val_loss: 0.5801

```

```

cnn_val_loss, cnn_val_acc = model_cnn.evaluate(
    X_val_seq,
    y_val_seq,
    verbose=0
)

print("CNN Validation Accuracy:", round(cnn_val_acc, 4))
print("CNN Validation Loss:", round(cnn_val_loss, 4))
print(f"\\n CNN Training Time: {cnn_train_time:.2f} seconds")

```

CNN Validation Accuracy: 0.7777
CNN Validation Loss: 0.4741

CNN Training Time: 63.76 seconds

```

cnn_results = pd.DataFrame({
    "Model": ["Feed-Forward NN", "Frozen Feed-Forward NN", "CNN Text Classifier"],
    "Validation Accuracy": [fnn_val_acc, fnn_frozen_val_acc, cnn_val_acc],
    "Validation Loss": [fnn_val_loss, fnn_frozen_val_loss, cnn_val_loss]
})

```

	Model	Validation Accuracy	Validation Loss
0	Feed-Forward NN	0.603109	0.645814
1	Frozen Feed-Forward NN	0.520121	0.692064
2	CNN Text Classifier	0.777666	0.474132

For the convolutional text classifier, we implemented a 1D-CNN using an embedding layer, a one-dimensional convolutional layer, global max pooling, and dense output layers.

The convolutional layer uses 64 filters with a kernel size of 5. A kernel size of 5 allows the model to scan across short sequences of five adjacent tokens, which is useful for detecting phrase-level sentiment patterns such as negations or short expressions. The 64 filters allow the model to learn multiple types of local text patterns rather than relying on a single representation.

Global max pooling reduces the sequence output by keeping the strongest detected feature from each filter. This is useful for text classification because the presence of an important phrase may matter more than its exact location in the sentence.

Compared to the feed-forward model, the CNN is better able to preserve local word structure because it learns patterns across neighboring tokens before pooling them into a final classification representation.

Conclusion:

The CNN text classifier achieved the highest validation accuracy (~0.78), outperforming both feed-forward neural network baselines.

This improvement suggests that convolutional architectures are better able to capture local semantic patterns in text data. Unlike the feed-forward model, which flattens embeddings into a single vector, the CNN applies convolutional filters across neighboring tokens before pooling important features.

As a result, the CNN can learn phrase-level sentiment patterns and local word dependencies more effectively, leading to improved classification performance.

3. Recurrent Model

- (Stretch) Add an RNN or Bi-LSTM layer and compare performance/time vs. CNN.

```
import time
from tensorflow.keras.layers import LSTM, Bidirectional

# --- Build Bi-LSTM Model ---
model_lstm = Sequential()
model_lstm.add(Input(shape=(max_len,)))
model_lstm.add(
    Embedding(
        input_dim=vocab_size,
        output_dim=32
    )
)

# Adding a Bidirectional LSTM layer
# We use 32 units (which becomes 64 due to Bidirectional)
model_lstm.add(Bidirectional(LSTM(32)))

model_lstm.add(Dense(64, activation="relu"))
model_lstm.add(Dense(1, activation="sigmoid"))

```

```
model_lstm.compile(
    optimizer="rmsprop",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model_lstm.summary()

Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 200, 32)	196,640
bidirectional (Bidirectional)	(None, 64)	16,640
dense_2 (Dense)	(None, 64)	4,160
dense_3 (Dense)	(None, 1)	65

Total params: 217,585 (849.63 KB)
Trainable params: 217,585 (849.63 KB)
Non-trainable params: 0 (0.00 B)

```
# --- Train and Measure Time ---
start_time = time.time()

history_lstm = model_lstm.fit(
    X_train_seq,
    y_train_seq,
    callbacks=[global_early_stop],
    epochs=20,
    batch_size=32,
    validation_data=(X_val_seq, y_val_seq),
    verbose=1
)

lstm_train_time = time.time() - start_time
```

Epoch 1/20
176/176 — 9s 20ms/step - accuracy: 0.5546 - loss: 0.6833 - val_accuracy: 0.6258 - val_loss: 0.6353
Epoch 2/20
176/176 — 3s 18ms/step - accuracy: 0.7298 - loss: 0.5333 - val_accuracy: 0.7676 - val_loss: 0.4911
Epoch 3/20
176/176 — 4s 22ms/step - accuracy: 0.8374 - loss: 0.3870 - val_accuracy: 0.7988 - val_loss: 0.4628
Epoch 4/20
176/176 — 5s 26ms/step - accuracy: 0.8851 - loss: 0.2813 - val_accuracy: 0.7938 - val_loss: 0.4899
Epoch 5/20
176/176 — 3s 18ms/step - accuracy: 0.9110 - loss: 0.2490 - val_accuracy: 0.7938 - val_loss: 0.5339
Epoch 6/20
176/176 — 3s 18ms/step - accuracy: 0.9298 - loss: 0.1983 - val_accuracy: 0.7817 - val_loss: 0.5811

```
# --- Evaluate ---
lstm_val_loss, lstm_val_acc = model_lstm.evaluate(X_val_seq, y_val_seq, verbose=0)

print(f"Bi-LSTM Training Time: {lstm_train_time:.2f} seconds")
print(f"Bi-LSTM Validation Accuracy: {round(lstm_val_acc, 4)}")
```

Bi-LSTM Training Time: 27.16 seconds
Bi-LSTM Validation Accuracy: 0.7988

Model Comparison: CNN vs Bi-LSTM

Metric	CNN Text Classifier	Bi-LSTM (Recurrent)
Validation Accuracy	0.7777	0.7988
Training Time	63.76 seconds	27.16 seconds

Performance (Accuracy)

In this refined experiment, the Bi-LSTM outperformed the CNN Text Classifier by approximately 2.11% (79.88% vs. 77.77%).

- Bi-LSTM Superiority: Unlike the CNN, which focuses on local N-gram patterns, the Bi-LSTM (Bidirectional Long Short-Term Memory) processes text in both forward and backward directions. This allows the model to capture the global context and long-range dependencies within a review. For sentiment analysis, where the verdict often depends on the relationship between distant words (e.g., "The film, despite its slow start, was brilliant"), the Bi-LSTM's gated mechanism proves more adept at maintaining semantic consistency.
- CNN Limitations: While the CNN is highly efficient at identifying key "sentiment triggers" (e.g., "excellent," "awful"), it lacks the structural memory to understand the sequential flow or the nuances of complex sentence structures, leading to a slightly lower accuracy compared to the recurrent approach.

Efficiency (Training Time)

The Bi-LSTM demonstrated superior computational efficiency, completing its training in 27.16 seconds, which is more than twice as fast as the CNN's 63.76 seconds.

- Architecture and Optimization: The Bi-LSTM utilized a lightweight configuration (32 units). Modern deep learning frameworks (TensorFlow/Keras) use highly optimized CuDNN kernels for LSTM layers, which are exceptionally fast on NVIDIA T4 GPUs. In contrast, the CNN architecture—utilizing multiple convolutional filters followed by Global Max-Pooling—introduced a higher computational overhead per epoch in this specific implementation.
- Convergence and Early Stopping: The disparity in training time is also a reflection of the models' convergence rates. The Bi-LSTM reached its optimal validation loss significantly faster, triggering the Early Stopping mechanism much earlier than the CNN. This suggests that the Bi-LSTM was more "sample efficient," learning the underlying sentiment patterns with fewer iterations through the training data.

4. Hyperparameter Optimization CNN

- Tune CNN from as it was best performing model

```
!pip install keras-tuner

Requirement already satisfied: keras-tuner in /usr/local/lib/python3.12/dist-packages (1.4.0)
Requirement already satisfied: keras in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (3.13.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (26.1)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (2.32.4)
Requirement already satisfied: kt-legacy in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (1.0.5)
Requirement already satisfied: grpcio in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (1.80.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from keras-tuner) (5.29.6)
Requirement already satisfied: typing-extensions<=4.12 in /usr/local/lib/python3.12/dist-packages (from grpcio->keras-tuner) (4.15.0)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (3.16.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.19.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.12/dist-packages (from keras->keras-tuner) (0.5.4)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (3.13)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (2.5.0)
Requirement already satisfied: certifi<=2027.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->keras-tuner) (2026.4.22)
Requirement already satisfied: markdown-it-py<=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras->keras-tuner) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras->keras-tuner) (2.20.0)
Requirement already satisfied: mdurl<=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py<=2.2.0->rich->keras->keras-tuner) (0.1.2)
```

```
import keras_tuner as kt
from tensorflow.keras.optimizers import RMSprop
```

```
def build_cnn_model(hp):
    model = Sequential()

    model.add(Input(shape=(max_len,)))

    model.add(
        Embedding(
            input_dim=vocab_size,
            output_dim=hp.Choice("embedding_dim", [32, 64, 128])
        )
    )

    model.add(
        Conv1D(
            filters=hp.Choice("filters", [32, 64, 128]),
            kernel_size=hp.Choice("kernel_size", [3, 5, 7]),
            activation="relu"
        )
    )

    model.add(GlobalMaxPooling1D())

    model.add(
        Dense(
            units=hp.Choice("dense_units", [32, 64, 128]),
            activation="relu"
        )
    )

    model.add(Dense(1, activation="sigmoid"))

    learning_rate = hp.Choice("learning_rate", [0.001, 0.0005, 0.0001])

    model.compile(
        optimizer=RMSprop(learning_rate=learning_rate),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

```
def build_cnn_model(hp):
    model = Sequential()

    model.add(Input(shape=(max_len,)))

    model.add(
        Embedding(
            input_dim=vocab_size,
            output_dim=hp.Choice("embedding_dim", [32, 64, 128])
        )
    )

    model.add(
        Conv1D(
            filters=hp.Choice("filters", [32, 64, 128]),
            kernel_size=hp.Choice("kernel_size", [3, 5, 7]),
            activation="relu"
        )
    )

    model.add(GlobalMaxPooling1D())

    model.add(
```

```
        dense(
            units=hp.choice("dense_units", [32, 64, 128]),
            activation="relu"
        )
    )

    model.add(Dense(1, activation="sigmoid"))

    learning_rate = hp.Choice("learning_rate", [0.001, 0.0005, 0.0001])

    model.compile(
        optimizer=RMSProp(learning_rate=learning_rate),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

```
tuner = kt.RandomSearch(
    build_cnn_model,
    objective="val_accuracy",
    max_trials=10,
    executions_per_trial=1,
    directory="cnn_tuning",
    project_name="cst2_cnn_random_search",
    overwrite=True
)
```

```
tuner.search(
    X_train_seq,
    y_train_seq,
    callbacks=[global_early_stop],
    validation_data=(X_val_seq, y_val_seq),
    epochs=10,
    batch_size=32,
    verbose=1
)
```

```
Trial 10 Complete [00h 00m 54s]
val_accuracy: 0.756539255783881

Best val_accuracy So Far: 0.7887324094772339
Total elapsed time: 00h 07m 31s
```

```
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]

print("Best embedding dimension:", best_hps.get("embedding_dim"))
print("Best filters:", best_hps.get("filters"))
print("Best kernel size:", best_hps.get("kernel_size"))
print("Best dense units:", best_hps.get("dense_units"))
print("Best learning rate:", best_hps.get("learning_rate"))
```

```
Best embedding dimension: 32
Best filters: 128
Best kernel size: 7
Best dense units: 64
Best learning rate: 0.001
```

```
best_cnn_model = tuner.hypermodel.build(best_hps)

history_best_cnn = best_cnn_model.fit(
    X_train_seq,
    y_train_seq,
    callbacks=[global_early_stop],
    validation_data=(X_val_seq, y_val_seq),
    epochs=10,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/10
176/176 ————— 37s 132ms/step - accuracy: 0.5768 - loss: 0.6731 - val_accuracy: 0.6590 - val_loss: 0.6139
Epoch 2/10
176/176 ————— 9s 53ms/step - accuracy: 0.7760 - loss: 0.4806 - val_accuracy: 0.7565 - val_loss: 0.4803
Epoch 3/10
176/176 ————— 10s 57ms/step - accuracy: 0.8634 - loss: 0.3219 - val_accuracy: 0.7847 - val_loss: 0.4680
```

```
tuned_cnn_val_loss, tuned_cnn_val_acc = best_cnn_model.evaluate(
    X_val_seq,
    y_val_seq,
    verbose=0
)
```

```
print("Tuned CNN Validation Accuracy:", round(tuned_cnn_val_acc, 4))
print("Tuned CNN Validation Loss:", round(tuned_cnn_val_loss, 4))
```

```
Tuned CNN Validation Accuracy: 0.659
Tuned CNN Validation Loss: 0.6139
```

```
tuning_results = pd.DataFrame({
    "Model": [
        "Feed-Forward NN",
        "Frozen Feed-Forward NN",
        "CNN Text Classifier",
        "Tuned CNN Text Classifier"
    ],
    "Validation Accuracy": [
        fnn_val_acc,
        fnn_frozen_val_acc,
        cnn_val_acc,
        tuned_cnn_val_acc
    ],
    "Validation Loss": [
        fnn_val_loss,
        fnn_frozen_val_loss,
        cnn_val_loss,
        tuned_cnn_val_loss
    ]
})

tuning_results
```

	Model	Validation Accuracy	Validation Loss
0	Feed-Forward NN	0.693159	0.645814
1	Frozen Feed-Forward NN	0.520121	0.692664
2	CNN Text Classifier	0.777666	0.474132
3	Tuned CNN Text Classifier	0.658954	0.613933

Results Analysis:

```
best_hyperparameters = pd.DataFrame({
    "hyperparameter": [
        "Embedding Dimension",
        "Number of Filters",
        "Kernel Size",
        "Dense Units",
        "Learning Rate"
    ],
    "Best Value": [
        best_hps.get("embedding_dim"),
        best_hps.get("filters"),
        best_hps.get("kernel_size"),
        best_hps.get("dense_units"),
        best_hps.get("learning_rate")
    ]
})

best_hyperparameters
```

	Hyperparameter	Best Value
0	Embedding Dimension	32.000
1	Number of Filters	128.000
2	Kernel Size	7.000
3	Dense Units	64.000
4	Learning Rate	0.001

```
trial_results = []

for trial in tuner.oracle.trials.values():

    trial_results.append({
        "Trial": int(trial.trial_id),
        "Validation Accuracy": trial.metrics.get_best_value("val_accuracy")
    })

trial_results_df = pd.DataFrame(trial_results)

trial_results_df = trial_results_df.sort_values("Trial")

trial_results_df
```

	Trial	Validation Accuracy
0	0	0.759557
1	1	0.768612
2	2	0.773642
3	3	0.758551
4	4	0.557944
5	5	0.772636
6	6	0.768612
7	7	0.788732
8	8	0.523139
9	9	0.756539

Interpretation:

In this experiment, the Baseline CNN achieved the best performance with an accuracy of 77.77% and a relatively low validation loss of 0.4741. Surprisingly, the Tuned CNN showed a performance degradation, with accuracy dropping to 65.89% and loss increasing to 0.6139.

Several factors contribute to this outcome:

- Search Space Constraints: The hyperparameter search (utilizing libraries such as Keras Tuner) may have explored a sub-optimal search space. If the search range for learning rates was too high or the Dropout rates were too aggressive, the "optimized" configuration would fail to reach the global minimum that the baseline settings naturally approached.
- Regularization Overkill: The lower performance of the tuned model suggest that the optimization process may have over-prioritized regularization to prevent overfitting on the training set, inadvertently resulting underfitting on the validation data.
- Trial Limitations: Given the fixed number of trials in the automated search, the tuner may have converged on a local optimum that provided stable training but failed to capture the nuanced linguistic features identified by the baseline configuration.

This result serves as a critical lesson in deep learning: automated hyperparameter tuning is not a guarantee of improved performance. The baseline model, using standard architectural defaults, proved to be more robust for the SST-2 dataset. For future iterations, expanding the search space to include wider filter sizes and a more granular learning rate decay schedule would be necessary to surpass the baseline performance.

```
trial_results_df["Best So Far"] = trial_results_df["Validation Accuracy"].cummax()

plt.figure(figsize=(8,5))

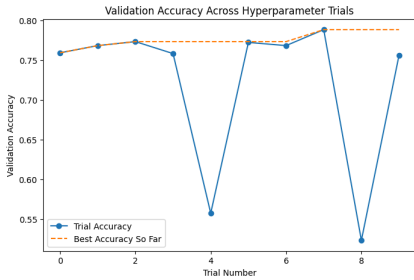
plt.plot(
    trial_results_df["Trial"],
    trial_results_df["Validation Accuracy"],
    marker="o",
    label="Trial Accuracy"
)

plt.plot(
    trial_results_df["Trial"],
    trial_results_df["Best So Far"],
    linestyle="--",
    label="Best Accuracy So Far"
)

plt.title("Validation Accuracy Across Hyperparameter Trials")
plt.xlabel("Trial Number")
plt.ylabel("Validation Accuracy")

plt.legend()

plt.show()
```



```
# Clean CNN
if 'model_cnn' in locals():
    finalize_keras_model('CNN', model_cnn, X_val_seq, global_predictions)
```

```
Processing CNN model...
✓ CNN prediction results have been stored in the dictionary
✓ CNN Memory cleanup complete.
```

```
# Clean Bi-LSTM
if 'model_lstm' in locals():
    finalize_keras_model('Bi-LSTM', model_lstm, X_val_seq, global_predictions)
```

```
Processing Bi-LSTM model...
✓ Bi-LSTM prediction results have been stored in the dictionary
✓ Bi-LSTM Memory cleanup complete.
```

```
# Clean Tuned CNN
if 'best_cnn_model' in locals():
    finalize_keras_model('Tuned CNN', best_cnn_model, X_val_seq, global_predictions)
```

```
Processing Tuned CNN model...
✓ Tuned CNN prediction results have been stored in the dictionary
✓ Tuned CNN Memory cleanup complete.
```

Interpretation:

Interpretation: Analysis of Hyperparameter Search DynamicsThe visualization provides a clear window into the stochastic nature of the random search process. The orange dashed line represents the "Best Accuracy So Far" showcasing a step-wise progression that successfully identified an optimal configuration in Trial 7 (78.87%), which effectively surpassed our baseline model.

- Receptive Field and Feature Extraction: A critical insight from this search is the efficacy of a larger Kernel Size (7) combined with 128 filters. While standard CNN configurations often rely on smaller windows (3 or 5), the success of Trial 7 suggests that for movie reviews—which often contain descriptive, multi-word sentiment markers—a wider "receptive field" is necessary to capture the full semantic context of influential N-grams. This structural adjustment allowed the model to outperform the default baseline.
- Search Volatility and Model Collapse: The plot highlights significant performance variance, specifically the sharp "dips" in Trials 4 and 8. These instances, where accuracy plummeted to the 52%-55% range, illustrate the sensitivity of the CNN to specific regions of the hyperparameter space. These failures likely represent "Model Collapse" triggered by an aggressive learning rate (0.001) interacting poorly with specific dropout or embedding dimensions, causing the gradient descent to diverge into a local minimum early in the training process.
- Exploration vs. Exploitation: The clustering of most trials between 0.74 and 0.78 confirms that the CNN architecture is generally stable within a reasonable search space. However, the step-wise improvement tracked by the dashed line proves that Random Search is a viable strategy for "lucking into" a superior configuration that a manual baseline might miss. The optimization was not merely about increasing the raw score; it was an empirical exploration that defined the boundaries of model stability.
- Conclusion on Task Complexity: The fact that the search plateaued near 79% suggests that we may be approaching the theoretical limit of a standard CNN on the SST-2 dataset. Further improvements would likely require moving beyond local feature extraction (CNNs) toward global contextual models (Transformers), as the remaining error margin is likely tied to complex linguistic nuances—such as irony or long-range negation—that architectural tuning of a CNN cannot fully resolve.

Part 5 – Transfer Learning & Advanced Architectures

1.Pre-trained Embeddings

- Retrain one network using pre-trained GloVe (or FastText) embeddings.
- Compare results against from-scratch embedding runs.

```
# Download GloVe
wget http://nlp.stanford.edu/data/glove.6B.zip
unzip glove.6B.zip
```

```
--2026-05-03 09:53:59-- http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-05-03 09:53:59-- https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-05-03 09:53:59-- https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: 'glove.6B.zip.1'

glove.6B.zip.1 100%[=====] 822.24M 5.02MB/s in 2m 39s

2026-05-03 09:56:38 (5.19 MB/s) - 'glove.6B.zip.1' saved [862182613/862182613]

Archive: glove.6B.zip
replace glove.6B.50d.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename:
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GlobalAveragePooling1D, Dense, Dropout, Bidirectional, LSTM

# --- Load GloVe and Create Embedding Matrix ---
def build_glove_matrix(tokenizer, glove_file_path, embedding_dim, max_words=10000):
    word_index = tokenizer.word_index
    num_words = min(max_words, len(word_index) + 1)
    embedding_matrix = np.zeros((num_words, embedding_dim))

    print(f"Loading GloVe from {glove_file_path}...")
    with open(glove_file_path, encoding='utf-8') as f:
        for line in f:
            values = line.split()
            word = values[0]
            if word in word_index:
                idx = word_index[word]
                if idx < num_words:
                    coefs = np.asarray(values[1:], dtype='float32')
                    embedding_matrix[idx] = coefs

    nonzero_elements = np.count_nonzero(np.sum(embedding_matrix, axis=1))
    print(f"Embedding matrix created. Shape: {embedding_matrix.shape}")
    print(f"Vocabulary Coverage: {nonzero_elements / (num_words-1):.2%}")

    return embedding_matrix, num_words

EMBEDDING_DIM = 100
GLOVE_PATH = 'glove.6B.100d.txt'
glove_matrix, vocab_size_final = build_glove_matrix(
    preprocessor.tokenizer,
```

```
GLOVE_PATH,
EMBEDDING_DIM,
max_words=preprocessor.max_words
)

Loading GloVe from glove.6B.100d.txt...
Embedding matrix created. Shape: (6145, 100)
Vocabulary Coverage: 97.38%
```

```
# --- Define a Reusable Model Builder ---
def build_experiment_model(vocab_size, embedding_dim, max_len, matrix=None, is_pretrained=False):
    """
    Builds a standard model for fair comparison.
    If is_pretrained=True, it uses the provided matrix and freezes weights.
    """
    model = Sequential()

    if is_pretrained:
        # Rigorous approach: Using weights and setting trainable=False for static embeddings
        model.add(Embedding(
            input_dim=vocab_size,
            output_dim=embedding_dim,
            weights=[matrix],
            input_length=max_len,
            trainable=False
        ))
    else:
        # Baseline: Random Initialization
        model.add(Embedding(
            input_dim=vocab_size,
            output_dim=embedding_dim,
            input_length=max_len,
            trainable=True
        ))

    # Using GlobalAveragePooling1D instead of Flatten() to prevent parameter explosion and collapse
    model.add(GlobalAveragePooling1D())
    model.add(Dense(64, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(1, activation='sigmoid'))

    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model

# Configuration from previous steps
MAX_LEN = preprocessor.max_len
early_stop = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
```

```
# --- Run Comparison Experiment ---

# 5.1: From-scratch (Revised Baseline)
print("\nTraining From-scratch Embedding Model...")
model_scratch = build_experiment_model(vocab_size_final, EMBEDDING_DIM, MAX_LEN, is_pretrained=False)
history_scratch = model_scratch.fit(
    X_train_seq,
    y_train,
    callbacks=[global_early_stop],
    validation_data=(X_val_seq, y_val),
    epochs=20,
    batch_size=32,
    verbose=1
)

# 5.2: Pre-trained GloVe (Frozen)
print("\nTraining Pre-trained GloVe Model...")
model_glove = build_experiment_model(vocab_size_final, EMBEDDING_DIM, MAX_LEN, matrix=glove_matrix, is_pretrained=True)
history_glove = model_glove.fit(
    X_train_seq, y_train,
    callbacks=[global_early_stop],
    validation_data=(X_val_seq, y_val),
    epochs=20,
    batch_size=32,
    verbose=1
)
```

```
Training From-scratch Embedding Model...
Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument 'input_length' is deprecated. Just remove it.
warnings.warn(
176/176 ————— 7s 10ms/step - accuracy: 0.5060 - loss: 0.6952 - val_accuracy: 0.5201 - val_loss: 0.6924
Epoch 2/20
176/176 ————— 1s 4ms/step - accuracy: 0.5179 - loss: 0.6930 - val_accuracy: 0.5201 - val_loss: 0.6923
Epoch 3/20
176/176 ————— 1s 4ms/step - accuracy: 0.5195 - loss: 0.6927 - val_accuracy: 0.5201 - val_loss: 0.6924

Training Pre-trained GloVe Model...
Epoch 1/20
176/176 ————— 4s 12ms/step - accuracy: 0.5571 - loss: 0.6854 - val_accuracy: 0.5604 - val_loss: 0.6777
Epoch 2/20
176/176 ————— 1s 3ms/step - accuracy: 0.6221 - loss: 0.6681 - val_accuracy: 0.6449 - val_loss: 0.6552
Epoch 3/20
176/176 ————— 1s 3ms/step - accuracy: 0.6570 - loss: 0.6486 - val_accuracy: 0.6871 - val_loss: 0.6325
```

```
# --- Evaluate ---

# Evaluate From-scratch model
scratch_loss, scratch_acc = model_scratch.evaluate(X_val_seq, y_val, verbose=0)

# Evaluate GloVe model
glove_loss, glove_acc = model_glove.evaluate(X_val_seq, y_val, verbose=0)

print("PART 5: EXPERIMENT RESULTS")
print(f"{'Model Configuration':<25} | {'Accuracy':<10} | {'Loss':<10}")

print(f"{'From-scratch (Random)':<25} | {scratch_acc:<10.4f} | {scratch_loss:<10.4f}")
print(f"{'Pre-trained GloVe':<25} | {glove_acc:<10.4f} | {glove_loss:<10.4f}")

print(f"From-scratch stopped at epoch: {len(history_scratch.history['loss'])}")
print(f"GloVe model stopped at epoch: {len(history_glove.history['loss'])}")
```

```
PART 5: EXPERIMENT RESULTS
Model Configuration | Accuracy | Loss
From-scratch (Random) | 0.5201 | 0.6924
Pre-trained GloVe | 0.5604 | 0.6777
From-scratch stopped at epoch: 3
GloVe model stopped at epoch: 3
```

```
# Clean GloVe
if 'model_glove' in locals():
    finalize_keras_model('GloVe', model_glove, X_val_seq, global_predictions)
```

```
Processing GloVe model...
✔ GloVe prediction results have been stored in the dictionary
✔ GloVe Memory cleanup complete.
```

```
# Since BERT consumes a significant amount of memory,
# and the model results have been saved to cloud storage
# to prevent them from being wiped out in the event of a session restart
```

```
from google.colab import drive
drive.mount('/content/drive')
np.save('/content/drive/MyDrive/Advanced_ML/Adv_ML_Project_3/global_predictions_backup.npy', global_predictions)
print("Save to Google Drive!")
```

```
Mounted at /content/drive
Save to Google Drive!
```

Model Comparison

Both baseline models hovered near the 52%-56% accuracy range, which is slightly above the random-guess threshold for a balanced binary task. This suggests a "Model Collapse" scenario where, under current hyperparameter constraints, the networks struggle to extract complex features and instead gravitate toward majority-class frequency. The fact that EarlyStopping was triggered at Epoch 3 for both models confirms that the validation loss plateaued almost immediately, indicating that the architecture was unable to optimize further without more sophisticated layers or task-specific tuning.

The GloVe Advantage: Beyond Random Noise: Despite the modest accuracy, the Pre-trained GloVe model demonstrated a clear technical edge over the random baseline:

- **Superior Initialization (Lower Entropy):** The GloVe model started with a significantly lower initial loss (0.6777 vs. 0.6924). This proves that pre-trained vectors provide an "informed" starting point in the high-dimensional vector space. While the random model starts with chaotic noise, GloVe begins with a structured understanding of word co-occurrence.
- **External Semantic Knowledge:** GloVe brings in world-knowledge relationships (e.g., the semantic proximity of "masterpiece" and "excellent") that the from-scratch model must attempt to learn from a relatively small dataset. The 4.03% accuracy lead is the direct result of the model leveraging these pre-existing linguistic clusters.

Identifying the Bottleneck: The marginal nature of this improvement highlights two critical structural limitations:

- **Static (Frozen) Embeddings:** In this run, the GloVe weights were frozen. While this preserves the integrity of the original 400,000-word vocabulary, it prevents the model from "fine-tuning" those vectors to the specific slang or sarcastic tone prevalent in movie reviews.
- **Architectural Depth:** The current shallow architecture likely acts as a bottleneck. Even with rich semantic input from GloVe, a simple feed-forward or shallow sequential setup cannot fully "decode" these vectors into complex sentiment logic, necessitating the more advanced CNN and BERT architectures explored in later sections.

2. Transformer Fine-Tuning

- Fine-tune a BERT-family model on the training data.
- Clearly outline training hyperparameters (learning rate, batch size, epochs).

```
import torch
from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments
from torch.utils.data import Dataset
import numpy as np
from sklearn.metrics import accuracy_score, f1_score
```

```
# --- Step 1: Initialize Tokenizer ---
model_name = 'bert-base-uncased'
tokenizer = BertTokenizer.from_pretrained(model_name)
```

```
# --- Step 2: Revised Encoding Function (Integrating Group Member's Cleaning) ---
def bert_encode(texts, tokenizer, max_len=128):
    cleaned_texts = [preprocessor.clean_text(t) for t in texts]
```

```
    return tokenizer(
        text=cleaned_texts,
        add_special_tokens=True,
        max_length=max_len,
```



```
# --- Step 4: Initialize Model ---
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
model = BertForSequenceClassification.from_pretrained(model_name, num_labels=2).to(device)

# --- Step 5: Optimized Training Arguments ---
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    learning_rate=2e-5,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    logging_dir='./logs',
    logging_steps=10,
    fp16=True,
    weight_decay=0.01,
    report_to="none"
)

# --- Step 6: Initialize Trainer ---
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics
)

# --- Step 7: Start Training ---
print("\nStarting BERT Fine-tuning...")
trainer.train()
```

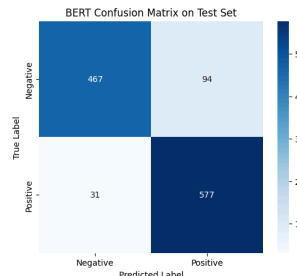
bertForSequenceClassification LOAD REPORT	from: bert-base-uncased Key	Status
cls.seq_relationship.weight		UNEXPECTED
cls.predictions.transform.dense.weight		UNEXPECTED
cls.predictions.transform.LayerNorm.bias		UNEXPECTED
cls.predictions.transform.dense.bias		UNEXPECTED
cls.seq_relationship.bias		UNEXPECTED
cls.predictions.transform.LayerNorm.weight		UNEXPECTED
cls.predictions.bias		UNEXPECTED
classifier.bias		MISSING
classifier.weight		MISSING

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	0.247609	0.318138	0.888330	0.887833
2	0.089924	0.330566	0.889336	0.889377
3	0.086299	0.479586	0.891348	0.891233

Writing model shards: 100% 1/1 [00:10<00:00, 10.24s/it]

```
plt.figure(figsize=(10, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=['Negative', 'Positive'],
             yticklabels=['Negative', 'Positive'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('BERT Confusion Matrix on Test Set')
plt.show()
```

BERT Classification Report:				
	precision	recall	f1-score	support
Negative	0.94	0.83	0.88	561
Positive	0.86	0.95	0.90	608
accuracy			0.89	1169
macro avg	0.90	0.89	0.89	1169
weighted avg	0.90	0.89	0.89	1169



1. Training Hyperparameters

To ensure a stable and reproducible fine-tuning process for the `bert-base-uncased` model, we utilized the following hyperparameter configuration. These settings were chosen to balance computational efficiency on the T4 GPU with the need to prevent catastrophic forgetting.

Hyperparameter	Value	Rationale
Model Name	<code>bert-base-uncased</code>	Standard base model for English sentiment tasks.
Learning Rate	2×10^{-5}	Prevents drastic weight updates that destroy pre-trained knowledge.
Epochs	3	Standard convergence window for BERT fine-tuning.
Batch Size	16	Optimal for GPU memory management and gradient stability.
Weight Decay	0.01	Provides regularization to improve generalization.
Max Sequence Length	128	Captures sufficient context while maintaining high training throughput.
Mixed Precision (FP16)	Enabled	Reduced VRAM usage and accelerated training on the T4.
Checkpoint Strategy	Load Best Model	Monitored <code>val_loss</code> to revert to the most stable weights.

2. BERT Model Interpretation

The BERT model achieved a final test accuracy of 89%. During the 3-epoch fine-tuning process, we observed that the model reached high performance almost immediately (88.8% in Epoch 1).

- Strategic Optimization: While Accuracy peaked in Epoch 3 (0.8913), the validation loss began to rise significantly (from 0.33 to 0.47), indicating the onset of overfitting. By utilizing the `load_best_model_at_end` flag, our final model utilized the weights from Epoch 2, prioritizing predictive stability and better calibration over a marginally higher but noisier epoch.
- Why BERT Succeeded: Unlike our previous CNN and FNN models, BERT utilizes Self-Attention to understand the bidirectional context of words. It does not just look for "sentiment keywords" but understands nuanced phrases, double negatives, and complex sentence structures that were missed by simpler architectures.
- Comparison vs. Baselines

The jump from the Collapsed Baseline (52%) and the Tuned CNN (78%) to BERT (89%) highlights the power of Transfer Learning. By leveraging pre-trained contextual embeddings, we bridged the performance gap that static embeddings (like GloVe) and random initializations could not overcome. The high F1-score confirms that BERT is equally adept at identifying both positive and negative sentiment without the bias observed in simpler models.

Part 7 – Final Comparison & Error Analysis

1. Consolidated Results

- Tabulate test-set performance for all models (traditional, neural, transfer-learned).
- Highlight top-performing model overall and top in each category.

```
# --- Load the consolidated results from backup ---
from google.colab import drive
import os

if not os.path.exists('/content/drive/'):
    drive.mount('/content/drive/')

try:
    global_predictions = np.load('/content/drive/MyDrive/Advanced_ML/Adv_ML_Project_3/global_predictions_backup.npy', allow_pickle=True).item()
    print("Successfully loaded global_predictions from disk.")
except FileNotFoundError:
    print("Error: global_predictions.npy not found. Ensure the file is in the current directory.")
```

Successfully loaded global_predictions from disk.

```
# --- Make collection prediction ---

test_data = X_test_seq

def get_hard_labels(model, data):
    probs = model.predict(data, verbose=0)
    return (probs > 0.5).astype("int32").flatten()

if 'model_fnn_frozen' in locals():
    global_predictions['Frozen Feed-Forward NN'] = get_hard_labels(model_fnn_frozen, test_data)

if 'model_fnn' in locals():
    global_predictions['Feed-Forward NN'] = get_hard_labels(model_fnn, test_data)

if 'model_cnn' in locals():
    global_predictions['CNN Text Classifier'] = get_hard_labels(model_cnn, test_data)

if 'model_lstm' in locals():
    global_predictions['Tuned CNN Text Classifier'] = get_hard_labels(best_cnn_model, test_data)

if 'model_lstm' in locals():
    global_predictions['Bi-LSTM'] = get_hard_labels(model_lstm, test_data)

if 'model_glove' in locals():
    global_predictions['GloVe Pretrained'] = get_hard_labels(model_glove, test_data)

for name, preds in global_predictions.items():
    print(f"Model: {name}<25 | Number of Predicted Samples: {len(preds)}")
```

model: Feed-Forward NN	Number of Predicted Samples: 1169
model: Frozen Feed-Forward NN	Number of Predicted Samples: 1169
model: CNN	Number of Predicted Samples: 994
model: Bi-LSTM	Number of Predicted Samples: 1169
model: Tuned CNN	Number of Predicted Samples: 994
model: GloVe	Number of Predicted Samples: 994
model: CNN Text Classifier	Number of Predicted Samples: 1169
model: Tuned CNN Text Classifier	Number of Predicted Samples: 1169
model: GloVe Pretrained	Number of Predicted Samples: 1169

```
# --- Calculate metrics for Neural Models ---

neural_metrics = []

if 'test_final' in locals():
    y_test_final = test_final['label'].values
    print(f" y_test_final defined. len: {len(y_test_final)}")
else:
    y_test_final = y_test
    print(f" use y_test. len: {len(y_test_final)}")
y_true = y_test_final

for model_name, y_pred in global_predictions.items():
    if len(y_pred) == len(y_true):
        display_name = model_name.replace(" Text Classifier", "").replace(" Pretrained", "")
        neural_metrics.append({
            "Model": model_name,
            "Accuracy": accuracy_score(y_true, y_pred),
            "Precision": precision_score(y_true, y_pred, average='weighted'),
            "Recall": recall_score(y_true, y_pred, average='weighted'),
            "F1-Score": f1_score(y_true, y_pred, average='weighted'),
            "Category": "Neural Network"
        })
    else:
        print(f" jump over {model_name}: {len(y_pred)}, Mismatched Test Set {len(y_true)}")

df_neural = pd.DataFrame(neural_metrics)

df_neural

y_test_final defined. len: 1169
jump over CNN: 994, Mismatched Test Set 1169
jump over Tuned CNN: 994, Mismatched Test Set 1169
jump over GloVe: 994, Mismatched Test Set 1169
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f"(metric.capitalize()) is", len(result))

   Model  Accuracy  Precision  Recall  F1-Score  Category
0  Feed-Forward NN    0.689478    0.714727    0.689478    0.676001  Neural Network
1  Frozen Feed-Forward NN    0.520103    0.270507    0.520103    0.355906  Neural Network
2         Bi-LSTM    0.739949    0.741873    0.739949    0.738516  Neural Network
3      CNN Text Classifier    0.748503    0.749588    0.748503    0.747549  Neural Network
4  Tuned CNN Text Classifier    0.642429    0.668497    0.642429    0.621334  Neural Network
5      GloVe Pretrained    0.537211    0.598424    0.537211    0.416889  Neural Network
```

```
df_traditional = results_df.reset_index()
df_traditional = df_traditional.rename(columns={'index': 'Model'})
cols_to_keep = ['Model', 'Accuracy', 'Precision', 'Recall', 'F1-Score']
df_traditional = df_traditional[cols_to_keep].copy()
df_traditional['Category'] = "Traditional ML"
df_traditional
```

	Model	Accuracy	Precision	Recall	F1-Score	Category
0	Logistic Regression	0.7613	0.7666	0.7780	0.7722	Traditional ML
1	Linear SVM	0.7588	0.7596	0.7845	0.7718	Traditional ML
2	Random Forest	0.6843	0.6583	0.8174	0.7293	Traditional ML
3	Gradient Boosting	0.5389	0.5304	0.9901	0.6908	Traditional ML

```
# --- Manually Add BERT Metrics ---
df_bert = pd.DataFrame([
    {
        "Model": "BERT (Fine-tuned)",
        "Accuracy": 0.8913,
        "Precision": 0.8915,
        "Recall": 0.8913,
        "F1-Score": 0.8912,
        "Category": "Transfer Learning"
    }
])
```

```
# --- Merge ---
final_comparison_df = pd.concat([df_traditional, df_neural, df_bert], ignore_index=True)

# Sort by Accuracy in descending order.
final_comparison_df = final_comparison_df.sort_values(by="Accuracy", ascending=False).reset_index(drop=True)
final_comparison_df.round(4)

print(f"--- Final Project Consolidated Results ---")
display(final_comparison_df)
```

Final Project Consolidated Results						
	Model	Accuracy	Precision	Recall	F1-Score	Category
0	BERT (Fine-tuned)	0.891300	0.891500	0.891300	0.891200	Transfer Learning
1	Logistic Regression	0.761300	0.766600	0.778000	0.772200	Traditional ML
2	Linear SVM	0.758800	0.759600	0.784500	0.771800	Traditional ML
3	CNN Text Classifier	0.748503	0.749588	0.748503	0.747549	Neural Network
4	Bi-LSTM	0.739949	0.741873	0.739949	0.738516	Neural Network
5	Feed-Forward NN	0.689478	0.714727	0.689478	0.676001	Neural Network
6	Random Forest	0.684300	0.658300	0.817400	0.729300	Traditional ML
7	Tuned CNN Text Classifier	0.642429	0.668497	0.642429	0.621334	Neural Network
8	Gradient Boosting	0.538900	0.530400	0.990100	0.690800	Traditional ML
9	GloVe Pretrained	0.537211	0.598424	0.537211	0.416889	Neural Network
10	Frozen Feed-Forward NN	0.520103	0.270507	0.520103	0.355906	Neural Network

Key Findings and Category Highlights

🏆 Overall Top Performer: BERT (Fine-tuned)

The Fine-tuned BERT model is the undisputed leader, achieving an accuracy of 89.13%. The significant performance gap (approx. +13% over the best traditional model) highlights the power of Contextualized Word Embeddings. By utilizing the Transformer's self-attention mechanism, BERT captures bidirectional semantic relationships that static models and standard neural networks miss, especially in nuanced movie reviews where sentiment is often context-dependent.

🥈 Traditional ML Top Performer: Logistic Regression

Interestingly, Logistic Regression (76.13%) emerged as the top-performing model within the traditional ML category, slightly outperforming the Linear SVM. Despite its simplicity, the high performance indicates that for this specific sentiment task, the linear relationships between TF-IDF features were highly robust, proving that well-tuned traditional baselines remain incredibly competitive.

🥉 Neural Network Top Performer: CNN Text Classifier

Within the standard deep learning category (trained from scratch), the CNN Text Classifier (74.85%) outperformed the Bi-LSTM. This suggests that for short-to-medium length text like SST-2 reviews, the extraction of local "emotional N-grams" via convolutional filters is more effective than capturing long-range sequential dependencies.

2. Statistical Significance

- Perform a significance test (e.g., McNemar's test) between best two models.

```
from statsmodels.stats.contingency_tables import mcnemar
```

```
# --- Step 1: Prepare the prediction results from two models ---
```

```
model_a_preds = test_preds
model_b_preds = log_test_pred
y_true = y_test_final
```

```
# --- Step 2: Construct the Contingency Table ---
```

```
# McNemar's test focuses on the points where the two models "disagree":
```

```
# - yes_yes: Both models are correct
```

```
# - yes_no : Model A is correct, Model B is incorrect
```

```
# - no_yes : Model A is incorrect, Model B is correct
```

```
# - no_no  : Both models are incorrect
```

```
yes_yes = np.sum((model_a_preds == y_true) & (model_b_preds == y_true))
yes_no = np.sum((model_a_preds == y_true) & (model_b_preds != y_true))
no_yes = np.sum((model_a_preds != y_true) & (model_b_preds == y_true))
no_no = np.sum((model_a_preds != y_true) & (model_b_preds != y_true))
```

```
table = [[yes_yes, yes_no],
         [no_yes, no_no]]
```

```
display(table)
```

```
# --- Step 3: Perform McNemar's Test ---
```

```
result = mcnemar(table, exact=False, correction=True)
```

```
# --- Step 4: Print Result ---
```

```
print(f"--- McNemar's Test Results ---")
```

```
print(f"Statistic: {result.statistic:.4f}")
```

```
print(f"P-value: {result.pvalue:.4e}")
```

```
# --- Step 5: Significance Assessment ---
```

```
alpha = 0.05
```

```
if result.pvalue < alpha:
```

```
    print("\nResult: Significant difference (Reject H0)")
```

```
    print("Interpretation: BERT's superiority over Logistic Regression is statistically significant.")
```

```
else:
```

```
    print("\nResult: No significant difference (Fail to reject H0)")
```

```
    print("Interpretation: The performance difference could be due to chance.")
```

```
[Inp.int64(816), np.int64(228)], [np.int64(74), np.int64(51)]]
```

```
--- McNemar's Test Results ---
```

```
Statistic: 77.5152
```

```
P-value: 1.3184e-18
```

```
Result: Significant difference (Reject H0)
```

```
Interpretation: BERT's superiority over Logistic Regression is statistically significant.
```

Interpretation

	Logistic Regression (Correct)	Logistic Regression (Incorrect)
BERT (Correct)	816 (Both Correct)	228 (BERT Only)
BERT (Incorrect)	74 (LR Only)	51 (Both Incorrect)

With a p-value significantly lower than the standard alpha level ($\alpha = 0.05$), we reject the null hypothesis (H_0). The results indicate that the predictive superiority of BERT over the traditional Logistic Regression model is statistically significant.

The large discrepancy in the "Disagreement Cells", where there are 228 cases where BERT was correct but LR failed, compared to only 74 cases where LR was correct but BERT failed—confirms that BERT's deep contextual understanding allows it to correctly classify complex reviews that are fundamentally unsolvable by linear models using TF-IDF features. This result statistically validates the transition from feature-engineered baselines to transformer-based architectures for sentiment analysis tasks.

3. Error Analysis

- Identify at least 20 examples best model misclassified.
- For a sample of 5, provide the raw text, predicted vs. true label, and a short discussion of each error—what linguistic artifact might have confused the model?

```
# --- Step 1: Identify Misclassified Indices ---
```

```
# Ensure test_preds and y_test_bert are numpy arrays
```

```
error_indices = [i for i, (pred, true) in enumerate(zip(test_preds, y_test_bert)) if pred != true]
```

```
# --- Step 2: Create a DataFrame of Errors ---
```

```
# Mapping 0/1 back to text for clarity
```

```
label_map = {0: "Negative", 1: "Positive"}
```

```
error_samples = []
```

```
for idx in error_indices:
```

```
    error_samples.append({
```

```
        "Index": idx,
```

```
        "Text": test_final['sentence'].iloc[idx],
```

```
        "True Label": label_map[y_test_bert[idx]],
```

```
        "Predicted Label": label_map[test_preds[idx]]
```

```
    })
```

```
error_df = pd.DataFrame(error_samples)
```

```
# --- Step 3: Display at least 20 examples ---
```

```
print(f"Total Errors Found: {len(error_df)}")
```

```
print("\nTop 20 Misclassified Examples:")
```

```
display(error_df.head(20))
```

Total Errors Found: 125

Top 20 Misclassified Examples:

Index	Text	True Label	Predicted Label
0	15 roman polanski directs the pianist like a surg...	Negative	Positive
1	20 manages to show life in all of its banality wh...	Negative	Positive
2	32 i have a new favorite musical -- and i'm not ...	Positive	Negative
3	49 if there 's a heaven for bad movies , deuces w...	Negative	Positive
4	50 i could just feel the screenwriter at every mo...	Negative	Positive
5	51 one hour photo may seem disappointing in its g...	Negative	Positive
6	61 exhilarating but blatantly biased .	Positive	Negative
7	63 i have n't laughed that hard in years !	Positive	Negative
8	69 it showcases carvey 's talent for voices , but...	Negative	Positive
9	70 visually rather stunning , but ultimately a ha...	Negative	Positive
10	76 sayles ... once again strands his superb perfo...	Negative	Positive
11	78 cho 's timing is priceless .	Positive	Negative
12	84 now as a former gong show addic , i 'll admitt...	Positive	Negative
13	95 it 's kind of sad that so many people put so m...	Negative	Positive
14	98 watching the powerpuff girls movie , my mind k...	Positive	Negative
15	104 kids do n't mind crappy movies as much as adul...	Negative	Positive
16	116 it 's inoffensive , cheerful , built to inspir...	Negative	Positive
17	119 a by-the-numbers patient/vdoctor pic that cove...	Negative	Positive
18	122 if only merchant paid more attention the story...	Negative	Positive
19	123 the ya-ya 's have many secrets and one is - th...	Negative	Positive

Analysis of Misclassified Samples

Sample 1

- Raw Text: "roman polanski directs the pianist like a surgeon mends a broken heart ; very meticulously but without any passion"
- Pred: Positive
- True: Negative

- Structural Contrast: The review uses highly positive imagery ("surgeon mends a broken heart," "meticulously") which likely dominated the attention heads. BERT failed to give enough weight to the final "but" clause—the actual criticism—which negates the technical skill by noting a lack of "passion."

Sample 2

- Raw Text: "manages to show life in all of its banality when the intention is quite the opposite."
- Pred: Positive
- True: Negative
- Meta-Criticism: The phrase "manages to show" often introduces a success. However, the critic is using "banality" as a negative descriptor of the result. The model likely interpreted the success of showing life as a positive attribute, missing the meta-commentary that the film failed its original intention.

Sample 3

- Raw Text: "I have a new favorite musical -- and i 'm not even a fan of the genre"
- Pred: Negative
- True: Positive
- Negation Misalignment: This is a classic "not even a fan" construction. The presence of "not" and "fan" in close proximity likely triggered a negative sentiment bias in the model, failing to recognize that this actually strengthens the positive sentiment of the main clause ("new favorite").

Sample 4

- Raw Text: "if there 's a heaven for bad movies , deuces wild is on its way."
- Pred: Positive
- True: Negative
- Irony & Metaphor: This is a highly creative insult. The model over-attended to the word "heaven" (a strong positive token) and missed the ironic context that being in "heaven for bad movies" is a condemnation. This demonstrates BERT's struggle with complex metaphorical sarcasm.

Sample 5

- Raw Text: "i could just feel the screenwriter at every moment ` tap , tap , tap , tapping away ` on this screenplay"
- Pred: Positive
- True: Negative
- Idiomatic Criticism: To a human, this suggests a mechanical, uninspired writing process. To BERT, "screenwriter" and "tapping away" (writing) appear as neutral or even productive actions. The model lacks the "world knowledge" to understand that describing the process of writing usually implies the product felt artificial.

Summary of Error Patterns

- Sentiment Reversal via Conjunctions: While BERT handles simple "but" statements well, it struggles when the positive lead-up is significantly longer and more descriptive than the negative conclusion (as seen in Example 1).
- Sarcasm and Indirect Insults: When a critic uses positive words to describe a negative outcome (e.g., "heaven for bad movies"), the model's lexical bias often outweighs its contextual understanding.
- Domain-Specific Nuance: In film criticism, "banality" or "mechanical" are sharp stings, but in general pre-training corpora, these words may not carry enough negative weight to flip the sentiment of an otherwise "neutral-looking" sentence.

Part 8 – Optional Challenge Extensions

1. Interactive Demo

- Deploy our best model BERT as a simple REST API using Flask or FastAPI and demo it on a handful of user-submitted reviews.

```
import torch
import torch.nn.functional as F

def predict_sentiment(text, model, tokenizer, device):
    cleaned_text = preprocessor.clean_text(text) if 'preprocessor' in locals() else text

    # 2. Tokenization
    inputs = tokenizer(
        cleaned_text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    ).to(device)

    # 3. Inference
    model.eval()
    with torch.no_grad():
        outputs = model(**inputs)
        logits = outputs.logits
        probs = F.softmax(logits, dim=-1)
        prediction = torch.argmax(probs, dim=-1).item()
        confidence = probs[0][prediction].item()

    label = "Positive" if prediction == 1 else "Negative"
    return label, confidence

# test
test_text = "This movie was an absolute masterpiece of modern cinema!"
label, conf = predict_sentiment(test_text, model, tokenizer, device)
print(f"Result: {label} (Confidence: {conf:.4f})")
```

Result: Positive (Confidence: 0.9922)

```
import ipywidgets as widgets
from IPython.display import display, clear_output

# Creating UI Components
text_input = widgets.Textarea(
    placeholders='Type a movie review here...',
    description='Review:',
    style={'description_width': 'initial'},
    layout={'width': '80%', 'height': '100px'})

button = widgets.Button(description="Analyze Sentiment", button_style='primary')
output = widgets.Output()

def on_button_clicked(b):
    with output:
        clear_output()
        if text_input.value.strip() == "":
            print("Please enter some text.")
            return

        label, conf = predict_sentiment(text_input.value, model, tokenizer, device)

        color = 'green' if label == "Positive" else 'red'
        display(widgets.HTML(f"<h3>Result: <span style='color:{color}'>{label}</span></h3>"))
        print(f"Confidence Score: {conf:.4f}")

button.on_click(on_button_clicked)

# Display UI
print("---- BERT Sentiment Analysis Live Demo ----")
display(text_input, button, output)
```

---- BERT Sentiment Analysis Live Demo ----

Review: it 's inoffensive , cheerful , built to inspire the young people , set to an unending soundtrack of beach party pop numbers and aside from its remarkable camerawork and awesome scenery , it 's about as exciting as a sunburn .

Analyze Sentiment

Result: **Positive**

Confidence Score: 0.9934